

Practice 2: a lexer as a state machine

Languages and Compilers Design · Week 2

Practice 1 left you with a compiler that reads a line, guesses what it is, and builds IR for it. This practice puts a real front end in front of the builder: a lexer that walks the source byte by byte and turns it into a stream of typed tokens, and a wider syntax with types, `mut` and mandatory initialisers. Every later phase — the parser, the AST, the semantic checks — consumes the tokens you produce here.

Grading (3 points total)

Task 1 — the lexer is a hand-written state machine over bytes; every token carries its kind, text, line and column;

lexical errors are reported with `line:column`

2 pt

Task 2 — the new syntax is compiled through the builder API; `mut` and declaration-before-use are enforced with

`line:column`; tests/ has at least 5 valid and 5 invalid programs with their expected output

1 pt

Code generation stays in `llvmlite.ir` (or `llvm::IRBuilder`) exactly as in Practice 1: the only IR text is `str(module)`.

A compiler that assembles IR from strings gets 0 for Task 2. A lexer built on regular expressions, `split()` or a lexer generator gets 0 for Task 1.

YOUR REPOSITORY

From this practice on, your compiler lives in a Git repository that you create and own (GitHub or GitLab, private or public — your choice). Create it before you start Task 1, commit the Practice 1 code as the first commit, and commit at least once per task. Keep one layout for the whole course: `compiler.py` (or `compiler.cpp`) at the root, a `tests/` folder next to it, and a short `README.md` that says how to run the compiler and the tests. Work on each practice in its own branch (`practice-2` for this one) and open a pull request into `main` when you are done; add `vovaskochko` as a reviewer of the pull request. The Moodle submission for every practice and project stage is two links — the repository and the pull request — plus `ai_usage.txt`. Give the teacher read access if the repository is private. Merge the pull request after the review, so the next practice starts from `main`.

THE COMPILER INTERFACE

Keep the layout from Practice 1. The main file is `compiler.py` (or `compiler.cpp`). It takes two arguments and writes the IR once:

```
python3 compiler.py input.txt output.ll      # or: ./compiler input.txt output.ll
lli output.ll                               # run the IR directly while debugging
llc -filetype=obj -relocation-model=pic output.ll -o output.o
clang -fPIE output.o -o program && ./program
```

`lli` interprets the `.ll` without linking; it prints the same line as the binary and is the fastest way to check a change. A `full_compiler.sh` that does the last three steps in one call is welcome but optional.

On any error the compiler prints one line to `stderr` and exits with a non-zero code, without writing the output file:

```
compilation error: line 2:8: unexpected byte '$'
```

`line:column` is the position of the first byte of the offending token, both 1-based.

TASK 1 — THE LEXER (2 POINTS)

What a lexer does

The lexer reads the source as bytes, one at a time, and groups them into **tokens**: the smallest units the language has words for. It knows nothing about statements — that is the next phase. Its whole job is: which bytes belong together, what kind of thing they form, and where they were.

Implement it as a **state machine**: a loop that reads one byte, looks at the current state and the byte's class (letter, digit, space, {, ...), and either stays in the state, switches to another one, or emits a token and returns to the start state.

Worked example

Take the input `i32 mut x{ 10 }\\n var t`. The machine starts in the **initial** state, which classifies bytes. It reads three spaces and ignores them. Then it sees `i`: a letter, so a name or a keyword has begun — remember the byte and switch to the **identifier** state. That state reads `3` and `2`: alphanumeric, still part of the same word, now

i32. The next byte is a space, which ends the word. i32 is a built-in type name, so the token is a keyword, not an identifier; append it and return to the initial state. The initial state finds m, switches to the identifier state again, collects mut up to the next space — a keyword. Then x, and while in the identifier state the machine meets {: this ends the identifier x **and** is itself a token, so two tokens are appended. After the space, 1 is a digit: switch to the **number** state, collect 10, and end it on the space. Then } and the newline are single-byte tokens. On the next line var and t are identifiers.

The result is a sequence of typed tokens:

```
(i32, keyword, typename) (mut, keyword, specifier) (x, identifier)
({, block, start) (10, constant, numeric) (}, block, end) (\n, newline)
(var, identifier) (t, identifier)
```

Token categories

Category	Rule	Examples
keyword	a reserved word: a type name, a specifier, or a statement word	i32 mut exit
identifier	letters, digits and _, not starting with a digit, and not a keyword	x total_2
number	decimal digits only; no sign	0 10 255
block	the initialiser brackets	{ }
operator	assignment and the three arithmetic operators	:= + - *
end of line	\n; it terminates a statement	

Spaces and tabs separate tokens and are dropped. : alone, = alone, and every byte outside the table are lexical errors reported with line:column.

What the lexer must produce

- Each token is a record with at least: **kind**, **text**, **line**, **column**. Keep the position: every later error message is built from it.
- Keep the tokens in a vector, or in a vector of vectors — one inner vector per line. With one statement per line the second form makes the next step trivial.
- Keywords are recognised **after** the word is complete: the identifier state collects i32, mut, exit like any other word, and a lookup decides the kind.
- The code that decides whether a line of tokens is a valid statement, and then calls the builder, comes **after** the lexer and reads only tokens — never the source text.

Errors the lexer reports

- an unknown byte: line 2:8: unexpected byte '\$'
- { not closed on the same line: line 1:6: '{' is not closed before the end of the line
- a letter inside a number (10x), a : not followed by =

Rules

- The lexer is written by hand: a loop over bytes with an explicit state. No regular expressions, no str.split() / strtok, no lexer generators (flex, PLY, ANTLR).
- The scanner reads one byte at a time and may re-examine at most the byte that ended a token. It does not slice the line to look ahead.
- Bytes, not characters: the source is ASCII. A byte above 127 is an unknown byte.

Skeleton

```
class Token:
    def __init__(self, kind, text, line, col): ...

def lex(data: bytes):
    lines, tokens = [], []
    state, start, line, col = "START", 0, 1, 1
    i = 0
    while i <= len(data):
        b = data[i] if i < len(data) else None
        # one extra step: the end of input
```

```

if state == "START":
    if b is None: break
    elif b in (32, 9): pass # space, tab
    elif b == 10: lines.append(tokens); tokens = []; line += 1; col = 0
    elif is_alpha(b): state, start = "IDENT", i
    elif is_digit(b): state, start = "NUMBER", i
    elif b == ord("{"): tokens.append(Token("lbrace", "{", line, col))
    ...
    else: raise CompileError(f"line {line}:{col}: unexpected byte ...")
elif state == "IDENT":
    if b is not None and (is_alpha(b) or is_digit(b)): pass
    else:
        word = data[start:i]
        tokens.append(Token(KEYWORDS.get(word, "ident"), word.decode(), ...))
        state = "START"; continue # re-read this byte in START
elif state == "NUMBER": ...
i += 1; col += 1
if tokens: lines.append(tokens)
return lines

```

The skeleton shows the shape: one loop, one `state`, one byte per step, `continue` without advancing when the byte that ended a token must be read again. Fill in the states and remember the line and column of each token's first byte.

Warning. Test the lexer on its own before touching the builder: print the token list for a few inputs and read it. A wrong token kind or an off-by-one column is cheap to fix here and expensive to find once the error surfaces two phases later.

TASK 2 – THE SYNTAX (1 POINT)

The language

Still one statement per line, every variable a 32-bit integer.

- `i32 x{5}` — declaration of a **const** variable. `i32` is the type name; the initialiser in `{}` is mandatory.
- `i32 mut y{10}` — declaration of a **mutable** variable. `mut` comes after the type name. All variables are `const` unless marked `mut`.
- The initialiser is a constant, a variable, or one operation (+, -, *) on two constants or variables: `{10}`, `{x}`, `{y+10}`, `{x * y}`. Spaces inside `{}` are optional.
- `y := 5 · y := x · y := x + 3` — assignment, with the same right-hand side as an initialiser. Only a `mut` variable may be assigned.
- `exit y · exit 42` — prints Program exit with result `<value>` and ends the program. A constant or a variable; the last line.
- A variable is declared once, before its first use. Names are letters, digits and `_`, not starting with a digit; `i32`, `mut` and `exit` are reserved.

Valid:

```

i32 x{0}
i32 mut y{10}
i32 z{2+5}
i32 mut t { x + 10 }
t := t * z
exit t

```

Expected: Program exit with result 70.

Errors the compiler reports

Each with the `line:column` of the token that caused it:

- assignment to a variable that is not `mut`: line 2:1: cannot assign to 'x': it is not mut
- a variable used before its declaration, in an initialiser, an assignment or `exit`: line 1:7: variable 'y' is used before its declaration

- a declaration without an initialiser: `line 1:9: variable 'x' needs an initialiser in {}`
- a variable declared twice; a line that is not one of the three statements; extra tokens on a line; a program without `exit`.

The exact wording is yours; the `line:column` and the `exit` code are not.

Code generation

Nothing new. A declaration is an `alloca` plus a `store` of the initialiser; an assignment is a `store`; a variable read is a `load`; `exit` is the `printf` call and `ret`. The builder from Practice 1 does all of it — the difference is that the instructions are now driven by tokens, not by a guess about the line.

Tests

A `tests/` folder with at least 5 programs that compile and run, and at least 5 that must fail, each with its expected output or error message in a file next to it. Cover: a `const` and a `mut` declaration, an initialiser with an operation, `exit` of a constant, odd spacing and blank lines; an unknown byte, an unterminated `{`, an assignment to a `const`, a use before declaration, a missing initialiser.

SUBMISSION

Two parts, both by the end of the third day after the practice.

Show the teacher — during the practice, or within 3 days — from the `practice-2` branch of your repository: the token list your lexer prints for the worked example above; `compiler.py` (or `compiler.cpp`) with `tests/`; the valid program from Task 2 run through `lli` or `llc` and `clang` to `Program exit with result 70`; two failing tests with their `line:column` messages.

Submit in Moodle — the link to your repository, the link to the pull request from `practice-2` into `main` with `vovaskochko` added as a reviewer, and `ai_usage.txt` with your AI log. Merge the pull request after the review; Practice 3 builds the parser on these tokens, starting from `main`.